

OMNI Customer Portal — Integration & Data Logic

Companion to [PORTAL-FUNCTIONALITY.md](#). That doc says what each screen/flow *does*; this one says **where the data lives and which API calls each action makes**, so the portal can be paired with **Shopify** and a subscription app like **Loop Subscriptions** or **Recharge** without rewriting the UI.

Core principle: the portal is a **presentation layer**. It is *not* the source of truth. Shopify (plus the subscription app) owns the subscription state. Every button calls an **adapter operation**, the adapter calls the platform API, and the portal re-reads the result. Swap the adapter → same UI runs on Shopify-native, Loop, or Recharge.

Store this is built for (from the connected store): **OMNI Creatine**, [omnicreatine.com](#), **Shopify Plus**, USD, US/Central.

Accuracy note: Shopify Admin GraphQL names below are verified against the store's live schema/docs (**Admin API 2026-07**). Loop and Recharge are mapped at the **capability level** — confirm exact endpoint names in the Loop Developer API / Recharge API docs. The point is the *logic*, which is identical across providers.

1. Where subscriptions actually live

Shopify provides the subscription **substrate**; apps orchestrate it:

- **SubscriptionContract** — one per active subscription for a customer. Holds status, lines (product variants + quantities), delivery/billing policy, next billing date, delivery address, payment method. **This is the object every action mutates.**
- **SellingPlanGroup** → **SellingPlan** — defines *how* a product is sold on subscription: **billingPolicy** (bill every N weeks/months), **deliveryPolicy** (deliver every N), **pricingPolicy** (subscription discount). Cadence options (every 4/8/12 weeks, quarterly) are selling plans. Changing cadence = moving the contract line to a different selling plan.
- **Billing Cycles API** — auto-computed upcoming charges; used for skip, "order now", and editing *just the next* order vs. all future orders.

Loop / Recharge install on top of these: they own the contract lifecycle, add a merchant dashboard, dunning, bundling, prepaid, and hosted cancellation/retention flows — and write changes back to the Shopify contract. So our adapter either calls **Shopify Admin directly** (Shopify-native subscriptions) or calls **Loop/Recharge** (which then updates Shopify).

2. Canonical data model (portal-internal)

The portal renders from this normalized shape. The adapter maps it to/from the provider.

```

type PortalStatus = "active" | "paused" | "inactive" | "payment_failed";

interface Subscription {
  id: string; // provider contract/subscription id (gid or numeric)
  status: PortalStatus;
  lines: Array<{
    lineId: string;
    productTitle: string;
    variantId: string; // flavor / pouch-count variant
    variantTitle: string; // "Peach · 1 pouch"
    quantity: number;
    sellingPlanId: string; // encodes cadence (every 4w / quarterly ...)
    price: number;
  }>;
  cadenceLabel: string; // "Every 4 weeks" (derived from selling plan)
  nextBillingDate: string | null;
  resumeDate: string | null; // set when paused
  shippingAddress: Address;
  paymentMethod: PaymentRef; // token/last4 only – never raw PAN
  currencyCode: "USD";
}

```

Field mapping:

Portal field	Shopify (SubscriptionContract)	Loop	Recharge
<code>id</code>	contract <code>gid://shopify/SubscriptionContract/...</code>	subscription id	<code>subscription.id</code>
<code>status</code>	<code>status</code> enum (see §3)	subscription status	<code>status</code>
<code>lines[].variantId</code>	<code>lines.edges[].node.productVariant.id</code>	line variant	<code>shopify_variant_id</code>
<code>lines[].sellingPlanId</code>	<code>lines...sellingPlanId</code> / <code>deliveryPolicy</code>	frequency/plan	<code>order_interval_*</code>
<code>nextBillingDate</code>	<code>nextBillingDate</code>	next order date	<code>next_charge_scheduled_at</code>
<code>shippingAddress</code>	<code>deliveryMethod...address</code>	shipping address	<code>address</code>
<code>paymentMethod</code>	<code>customerPaymentMethod</code>	payment method	<code>payment_method</code>

3. Status mapping (the state model, at the data layer)

Functionality doc uses 3 states; Shopify has 5. Map them:

Portal state	Shopify SubscriptionContractSubscriptionStatus	Loop / Recharge	Portal behavior
<code>active</code>	<code>ACTIVE</code>	active	Full controls
<code>paused</code>	<code>PAUSED</code>	paused (Recharge: emulated — see §5)	Resume CTA + resume date
<code>inactive</code>	<code>CANCELLED</code> , <code>EXPIRED</code>	cancelled / expired	Winback + Restart
<code>payment_failed</code>	<code>FAILED</code>	dunning / payment error	Active with a "payment failed — update card" banner (do not hide controls)

Add the `payment_failed` state. Shopify moves a contract to `FAILED` after retries exhaust (dunning). The portal must surface "update your payment method" rather than silently showing it as active or cancelled.

4. The adapter interface

One interface, three implementations. UI depends only on this — never on a provider SDK.

```
interface SubscriptionProvider {
  listSubscriptions(customerId): Promise<Subscription[]>; // scoped to ONE customer
  getSubscription(id): Promise<Subscription>;

  pause(id, resumeDate?): Promise<Subscription>;
  resume(id): Promise<Subscription>;
  cancel(id, reason): Promise<Subscription>;
  reactivate(id): Promise<Subscription>; // or createFromLast()

  skipNextOrder(id): Promise<Subscription>;
  setNextOrderDate(id, date): Promise<Subscription>;
  chargeNow(id): Promise<OrderRef>; // "Order now"

  swapVariant(id, lineId, variantId): Promise<Subscription>; // flavor / pouch swap
  changeCadence(id, lineId, sellingPlanId): Promise<Subscription>; // e.g. quarterly
  addOneTime(id, variantId, opts): Promise<Subscription>; // gift / upsell, next order only
  applyDiscount(id, code): Promise<Subscription>;

  updateShipping(id, address): Promise<Subscription>;
  updatePaymentIntentUrl(id): Promise<Url>; // hosted card-update link

  createSubscription(customerId, input): Promise<Subscription>; // "Add product" = NEW contract
}
```

Every method takes an **id** → this is how per-subscription scoping (R2) is enforced.

5. Action → API mapping (the logic behind every button)

Portal action (see functionality doc)	Adapter op	Shopify Admin GraphQL (verified)	Loop / Recharge
Load portal	<code>listSubscriptions</code>	<code>customer { subscriptionContracts }</code>	list subscriptions by customer
Resume (paused→active)	<code>resume</code>	<code>subscriptionContractActivate(subscriptionContractId)</code>	activate / resume
Restart (inactive→active)	<code>reactivate</code> / <code>createSubscription</code>	<code>subscriptionContractActivate</code> if not <code>EXPIRED</code> ; else <code>subscriptionContractCreate</code> (or <code>subscriptionContractAtomicCreate</code>)	reactivate / create
Pause	<code>pause</code>	<code>subscriptionContractPause(subscriptionContractId)</code>	pause (Recharge: no native pause → skip/postpone or cancel+reactivate)
Cancel (final confirm only)	<code>cancel</code>	<code>subscriptionContractCancel(subscriptionContractId)</code>	cancel (fires Loop cancellation flow)
Skip next order	<code>skipNextOrder</code>	<code>subscriptionBillingCycleSkip(billingCycleInput)</code>	skip next charge
Next order date	<code>setNextOrderDate</code>	<code>subscriptionContractSetNextBillingDate(contractId, date)</code>	set next charge date
Order now	<code>chargeNow</code>	<code>subscriptionBillingAttemptCreate</code>	bill now
Swap flavor	<code>swapVariant</code>	<code>subscriptionContractUpdate</code> → <code>subscriptionDraftLineUpdate</code> → <code>subscriptionDraftCommit</code> (or <code>subscriptionContractProductChange</code>)	swap product/variant
Upgrade to quarterly / change cadence	<code>changeCadence</code>	draft: <code>subscriptionContractUpdate</code> → <code>subscriptionDraftLineUpdate</code> (new <code>sellingPlanId</code>) / <code>subscriptionDraftUpdate</code> → commit	change frequency
Add electrolytes / stack (recurring)	<code>swapVariant</code> +add line	draft: <code>subscriptionDraftLineAdd</code> → commit	add line
Free gift / one- time upsell (next order only)	<code>addOneTime</code>	<code>subscriptionBillingCycleContractEdit</code> (add line to the <i>next cycle</i> only)	add one-time product
Apply promo / savings offer	<code>applyDiscount</code>	draft: <code>subscriptionDraftDiscountCodeApply</code> / <code>subscriptionDraftDiscountAdd</code> → commit	apply discount
Update shipping	<code>updateShipping</code>	draft: <code>subscriptionDraftUpdate</code> (deliveryMethod/address) → commit	update address
Update / add payment	<code>updatePaymentIntentUrl</code>	<code>customerPaymentMethodGetUpdateUrl</code> / send update email (never handle raw PAN)	hosted payment update
Add product (Home)	<code>createSubscription</code>	<code>subscriptionContractCreate</code> → new contract	create new subscription

One-time vs recurring is a hard distinction (fixes the free-gift bug): - Recurring change → edit the **contract** (draft → commit).
Persists every order. - One-time (gift, single upsell) → edit the **next billing cycle only** (`subscriptionBillingCycleContractEdit`)

or add a Recharge **onetime**. It must **not** add a recurring line to the contract.

6. Per-subscription scoping (R2) — root cause & fix

Every mutation above takes a **single contract id**. A customer with two subscriptions has two `SubscriptionContract` gids. The current build's "cancel/reactivate hit all subscriptions" bug means it loops over all of the customer's contracts and calls the mutation on each.

Fix: the UI passes the selected `subscription.id` into the adapter; the adapter calls `subscriptionContractCancel(thatId)` — one id, one contract. Never iterate the customer's contract list for a single-subscription action. Render one card per contract, each with its own id and action row.

7. Segmentation & eligibility (quarterly gating + gift selection)

Offers are gated by customer/plan data — not shown to everyone (functionality §7).

Quarterly upgrade visibility (`show only to light subscribers`): - Read the contract's current line **quantity** (pouch count) and **selling plan interval**. - Rule: show if `pouchCount ∈ {1,2}` and cadence is monthly; **hide** if `pouchCount ≥ 3` or already on a quarterly selling plan. - Data source: Shopify `SubscriptionContract.lines` (quantity + sellingPlan) — or the equivalent line data from Loop/Recharge.

Free-gift selection (which one-time gift a customer sees): - Driven by segment: past purchases (`customer.orders`), **customer tags**, or a **metafield** (e.g. `custom.gift_tier`). "Did XYZ → free peach"; "did XY → other gift". - Data source: Shopify Customer `tags` / `metafields` / order history; Loop/Recharge customer attributes + tags. Compute eligibility server-side; the portal just renders the gift it's told the customer qualifies for.

Keep segmentation in the backend/adapter, not the React layer — the client should receive a resolved list of *eligible* offers, so business rules aren't shipped to the browser.

8. Cancellation & retention logic

The functionality doc's reason → retention-offer mapping (§6) is backed by real ops:

- Each retention action (**Skip / Pause / Change cadence / Apply discount / Swap**) calls the matching adapter op above — so "save" actions genuinely change the contract.
- `cancel()` is called **only** at the final confirmation step. Reaching a reason or a save screen never cancels.
- **Loop Subscriptions** has native **Cancellation Flows** (configurable retention: pause, skip, discount, swap, gift). Two options: (a) let Loop host the cancellation flow, or (b) keep our custom flow and call Loop ops per step, calling Loop `cancel` only at the end. **Recharge** offers equivalent retention/"cancellation prevention". Either way the logic is: *offer a real save action first; cancel only on explicit final confirm*.

9. Auth & session

The portal must authenticate the customer and scope all adapter calls to their id:

- **Shopify Plus (this store): New Customer Accounts + Customer Account API** (token identifies the customer), or an **App Proxy** (`/apps/portal`) with the Shopify-signed `logged_in_customer_id`. Admin subscription mutations run server-side with `write_own_subscription_contracts` scope — **never** expose Admin tokens to the browser.
- **Loop / Recharge:** issue a customer portal **session token**; the portal calls their customer-scoped API (or embeds/links their hosted portal for sensitive steps like payment).

10. Sync — the portal is not the source of truth

- After any mutation, **re-read** the subscription (mutation returns the updated object) and re-render. Optimistic UI is fine, but reconcile against the returned/refetched contract.
 - Subscribe to **webhooks** to stay in sync with events that happen outside the portal (renewals, dunning, merchant edits):
 - `subscription_contracts/create` , `subscription_contracts/update`
 - `subscription_billing_attempts/success` , `subscription_billing_attempts/failure`
 - `subscription_billing_cycle_edits/create|update|delete`
 - `orders/create` (fulfilled subscription orders → Order History)
 - Loop/Recharge emit equivalent webhooks; the adapter normalizes them to the model in §2.
-

11. Failure handling

- Every Shopify mutation returns `userErrors[]` — surface them; don't assume success.
 - A failed billing attempt → contract `FAILED` → portal `payment_failed` state (§3) with an "update payment" banner (dunning). This is a first-class state, not an edge case.
 - Use **idempotency** on charge/create ("Order now", "Add product") to avoid double-charging on retries.
 - Payment details are never handled in-portal — always a **hosted update URL** from the provider.
-

12. Rollout / provider-swap

1. Build the UI against the **adapter interface** (§4) only.
 2. Ship first with the **Shopify-native adapter** (Admin GraphQL, verified mutations above) **or** the **Loop adapter** if OMNI already runs Loop.
 3. To switch providers later, implement a new adapter — **zero UI changes**. The functionality doc's flows and the mapping table here are the contract.
-

13. Action → provider quick reference (one-liner recap)

- **Pause** → `subscriptionContractPause` · **Resume/Restart** → `subscriptionContractActivate`
- **Cancel** → `subscriptionContractCancel` (final confirm only)
- **Skip** → `subscriptionBillingCycleSkip` · **Next date** → `subscriptionContractSetNextBillingDate`
- **Order now** → `subscriptionBillingAttemptCreate`
- **Swap / cadence / add recurring** → `subscriptionContractUpdate` → `subscriptionDraft*` → `subscriptionDraftCommit`
- **One-time gift/upsell** → `subscriptionBillingCycleContractEdit` (next cycle only)
- **Discount** → `subscriptionDraftDiscountCodeApply` → commit
- **Add product** → `subscriptionContractCreate` (new contract)
- **All scoped to one contract id (R2). Loop/Recharge: same logic, their endpoints.**

Reference implementation (UI + flows this maps onto): <https://omni-customer-portal-delta.vercel.app>